

Module II

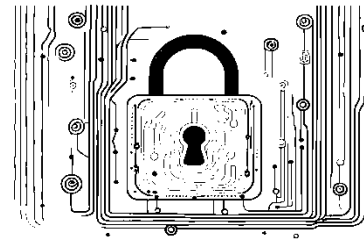
Chapter 1:

Conventional Symmetric Key Encryption



Identical Keys
(Symmetric)

Block Ciphers And Stream Ciphers



- Symmetric encryption ciphers consist of two main categories:
 - **Block Ciphers And Stream Ciphers.**
- Understanding the difference between a block cipher vs. stream cipher is kind of like the difference between watching a movie on DVD or via a streaming service. Sure, both will give you the entertainment you seek, but they each work differently in terms of mechanics and speed.
- Block and stream ciphers are two ways that you can encrypt data. Also known as **Bulk Ciphers**, they're two categories of Symmetric encryption algorithms.
- Block and stream ciphers are two separate routes to the same end goal of securing your data.

- **Encrypting information in chunks-** A **block cipher** breaks down plaintext messages into **fixed-size blocks** before converting them into cipher text using a key.
- **Encrypting information bit-by-bit-** A **stream cipher**, on the other hand, breaks a plaintext message down into **single bits**, which then are converted individually into cipher text using key bits.
- **Block ciphers** are the algorithms that form the backbone of many of the cryptographic technologies and processes that are **in use today in computer communications**.
- Where **stream ciphers** are concerned, you'll find stream ciphers in use in:
 - **SSL/TLS connections**
 - **Bluetooth connections**
 - **Cellular and 4G connections**

XOR Operations in Cryptography

- ‘ $\oplus$ ’ This is known as an **XOR**, or what stands for the “**exclusive - or**” logical operation.
- **Stream ciphers use bitwise XOR operators** on binary data as part of their encryption and decryption processes.
- So, what XOR does is compare two input bits and generate a single output bit. It’s a way to check for error among other things.
 - If both bits match, it returns a *0* (false); if they don’t match, it returns a *1* (true).

EX-OR (X-OR) Gate Truth Table

Inputs		Output $X = A \oplus B$
A	B	
0	0	0
0	1	1
1	0	1
1	1	0

What Is a Block Cipher?

- As the name would imply, a block cipher is one that allows you to use a key to encrypt data in big **blocks groups of a pre-determined size** (such as 128 bits, 256 bits, etc.).
- So, what this does is allow you to **encrypt all the data in a block as a single unit**.
- For example, let's say you have the message: "For the Alliance."
- If you want to encrypt that using a block cipher, the plaintext message would be broken down into smaller blocks of binary.
- So, "For the Alliance" would look something like this:

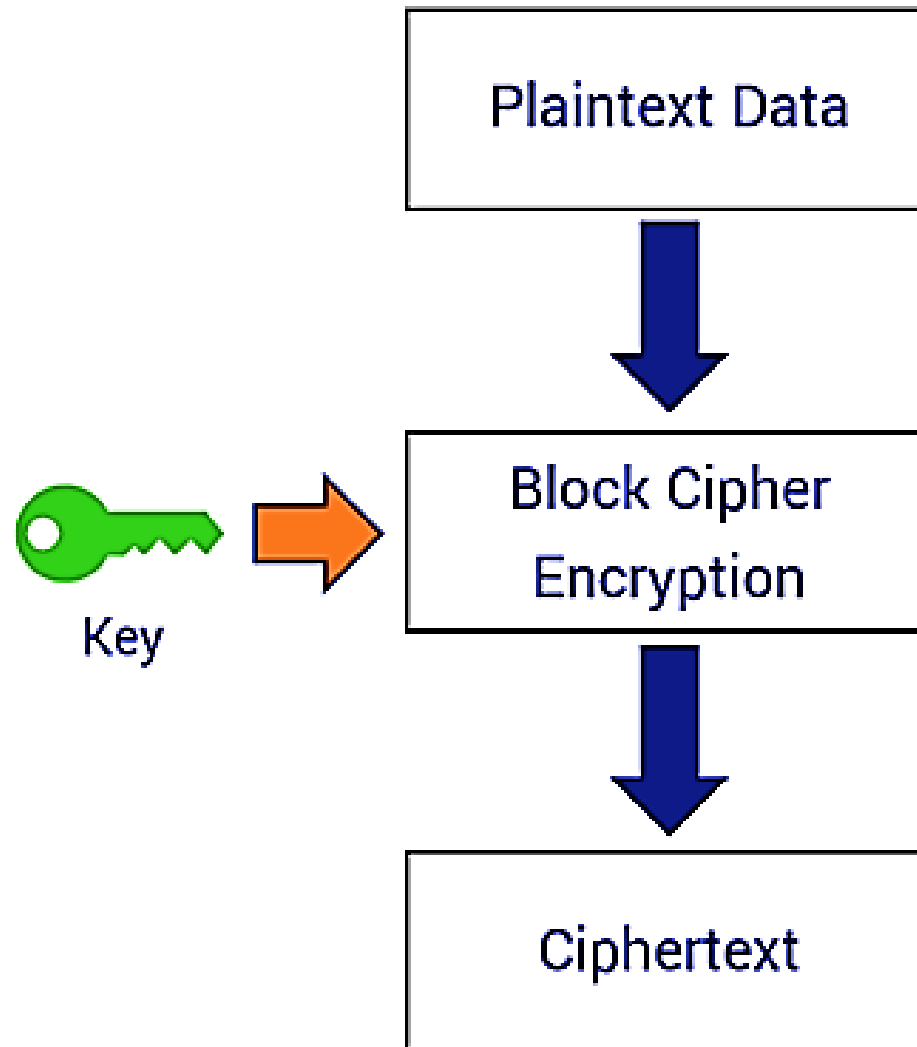
*010001100110111101110010001000000111010001101000011001010
010000001000001011011000110110001101001011000010110111001
1000110110010100101110*

- Now, if we break these 136 binary digits into blocks of, say, 4 digits, it divides nicely because 4 goes into 136 a total of 32 times. The blocks would look something like this:

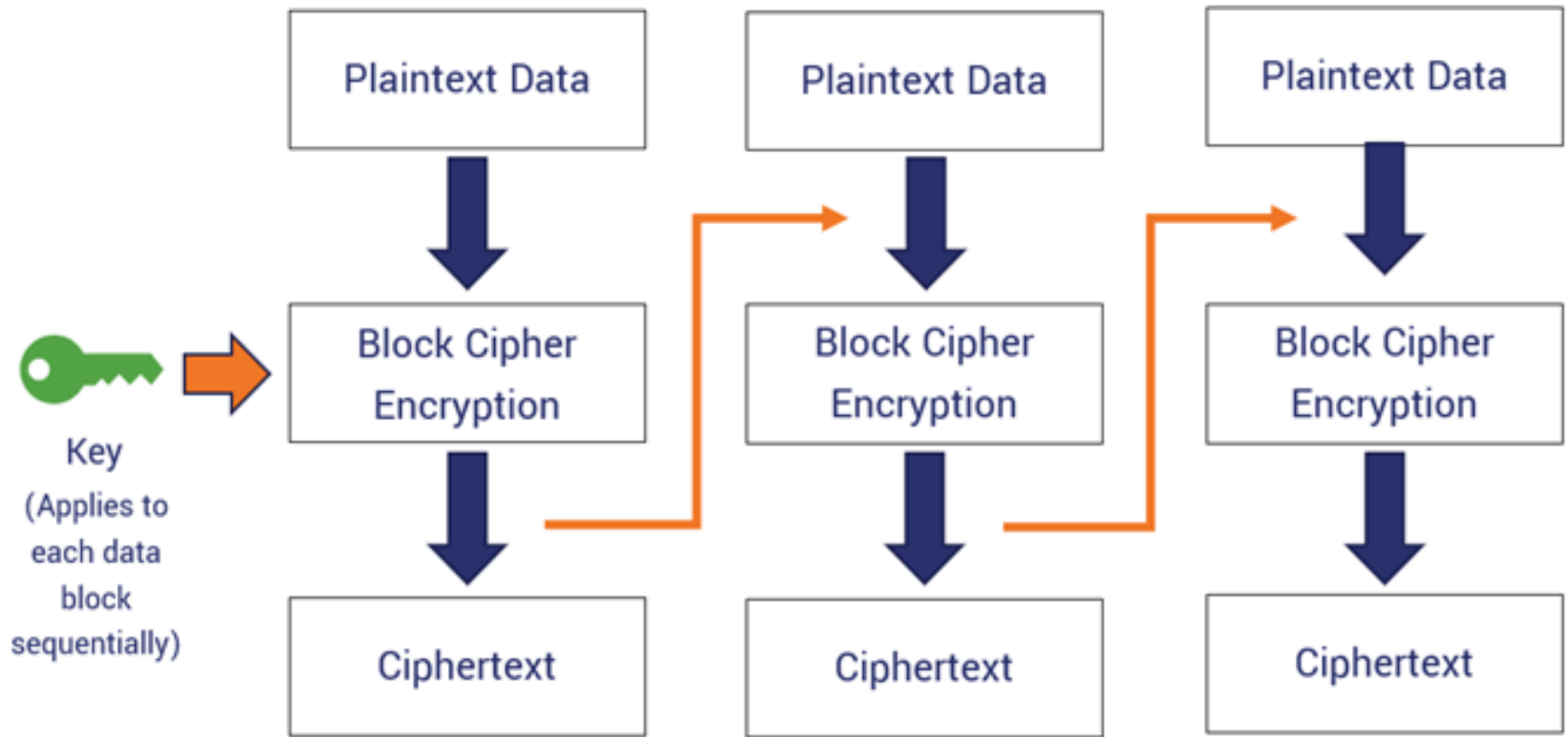


- But what do you do if there isn't enough data to form a complete block? Like, say, if you had 138 binary digits instead of 136 (meaning you'd have an incomplete last data block)?
- Then what you can do is add padding (this could be a set number of 0s) to the end of the block to ensure the block is that fixed size.

How a Basic Block Cipher Works



How a Basic Block Cipher Chaining Operation Works



Examples of Block Ciphers

- Data Encryption Standard (DES),
- Triple DES (3DES or TDEA),
- Advanced Encryption Standard (AES),
- International Data Encryption Algorithm (IDEA),
- Blowfish,
- Twofish,
- RC5

Advantages:

- Block Ciphers are difficult to mess with in terms of inserting symbols.
- They're highly analyzed and studied by cryptographers, which means that people smarter than you and me have attempted to break them. 😊

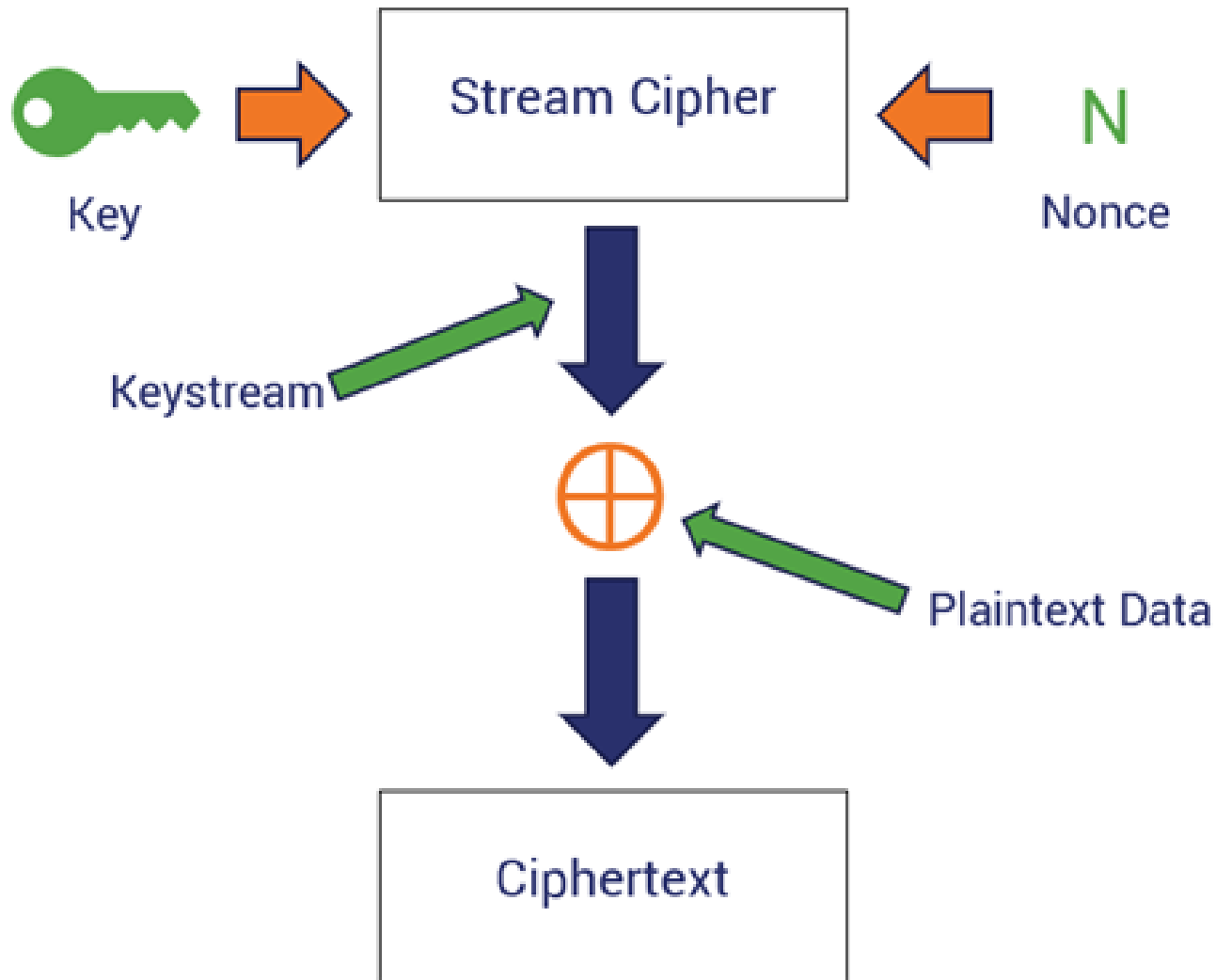
What Is a Stream Cipher?

- Unlike block ciphers, which require the formation of blocks prior to encryption, stream ciphers **encrypt data in long, pseudorandom streams**.
- Basically, this means you can process **one bit of data at a time** instead of waiting for a data block to form.
- Two types of stream ciphers:
 - **Synchronous stream ciphers (key auto-key, or KAK)**
These types of ciphers generate key streams independently of any previous plaintext or cipher texts.
 - **Self-synchronizing stream ciphers (asynchronous stream ciphers, cipher text auto key or CTAK)**
These ciphers, on the other hand, rely on previous cipher text bits to generate key streams.

How Stream Ciphers Work?

- A stream cipher encrypts data one bit at a time instead of in blocks. But a key part of this process is generating a stream of pseudorandom bits based on an encryption key and a seed/a nonce (a unique randomly generated number — “nonce” = number-only-used-once).
- Together, they create a key stream (that stream of pseudorandom bits we just mentioned) that gets XORed with your plaintext input, which encrypts it and results in your cipher text output.
- This rinse-and-repeat process happens over and over again with each bit of plaintext data.
- However, it's important to not use the same exact key-nonce combination again because it can result in a duplicate key stream.

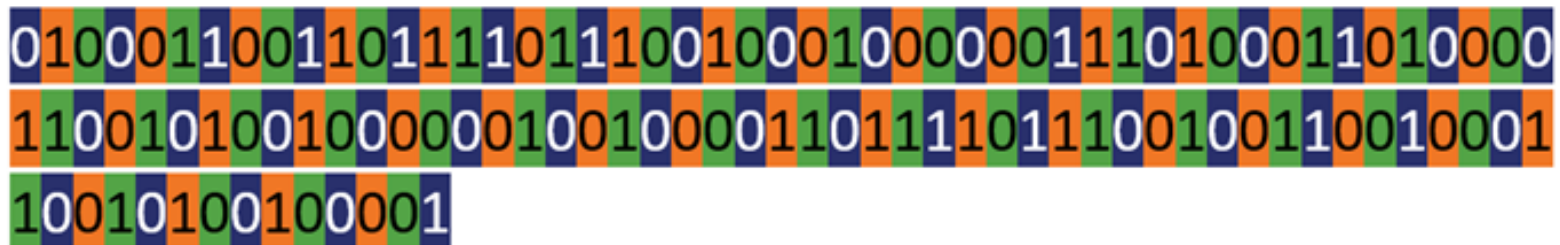
How a Basic Stream Cipher Works



- Example:
- This is a basic illustration to showcase the basic operations of a stream cipher.
- “For the People!”

01000110011011110111001000100000011101000110100001100101
00100000010010000110111101110010011001000110010100100001

- So, if we use a stream cipher to encrypt the message, we’ll do so one bit at a time.
- So, the plaintext data would be broken down into individual bits, which would look something like this:



0100011001101111011100100010000001110100011010000
1100101001000000100100001101111011100100110010001
10010100100001

Examples of Stream Ciphers:

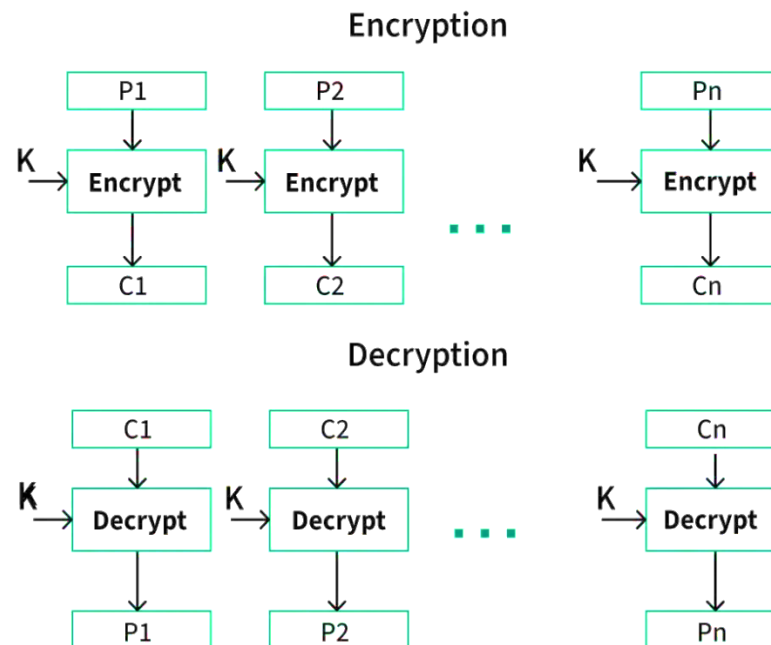
- **Salsa20** (software and hardware implementations),
- **ChaCha20** (the modified version of Salsa20; ChaCha20 is supported in TLS 1.3),
- **RC4** (for wireless networks),
- **A5** (for GSM cellular networks).

Block Cipher modes of Operation

- Block Cipher Modes of Operation define **how to securely encrypt and decrypt large amounts of data using a block cipher.**
- However, **to encrypt data larger than a single block, different modes of operation are used to ensure both security and efficiency.**
- Here are a few common modes:
 - **Electronic Code Book (ECB)**
 - **Cipher Block Chaining(CBC)**
 - **Cipher Feedback Mode (CFB)**
 - **Output Feedback Mode(OFB)**
 - **Counter Mode**

Electronic Code Book (ECB)

- The electronic codebook is the easiest block cipher mode of functioning.
- It is easier because of the **direct encryption of each block of input plaintext** and **output is in the form of blocks of encrypted cipher text**.
- Generally, if a message is larger than b bits in size, it can be **broken down into a bunch of blocks** and the **procedure is repeated**.



Advantages:

- **Parallel encryption** of blocks of bits is possible, thus it is a **faster way of encryption**.
- **Simple** way of the block cipher.

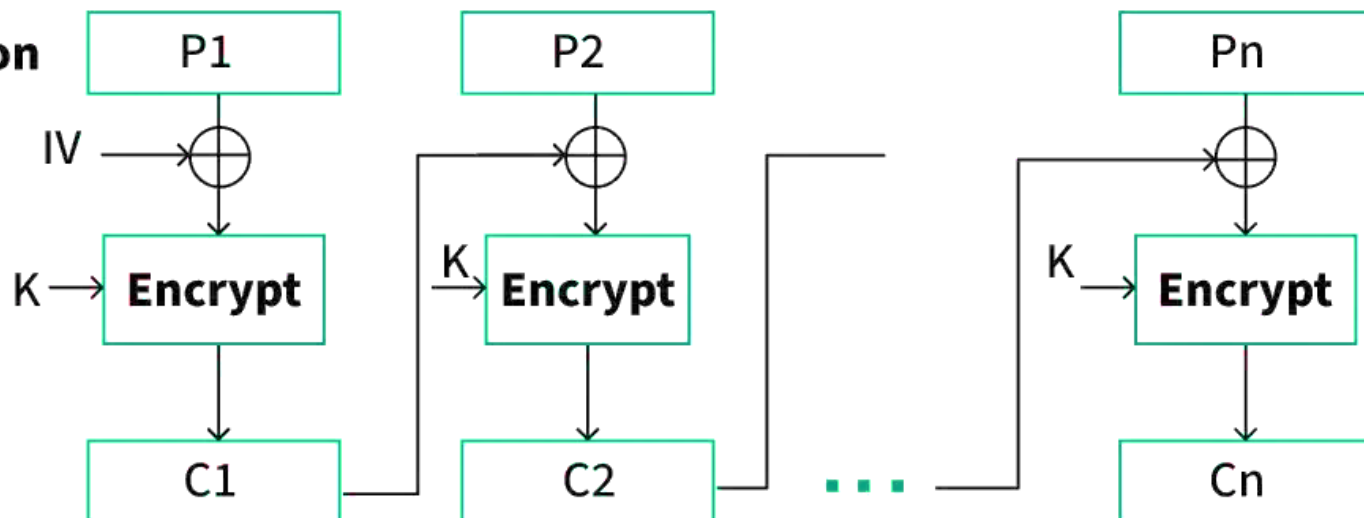
Disadvantages:

- Prone to cryptanalysis since there is a direct relationship between plaintext and cipher text.
- Identical plaintext blocks produce identical cipher text blocks, which can reveal patterns.

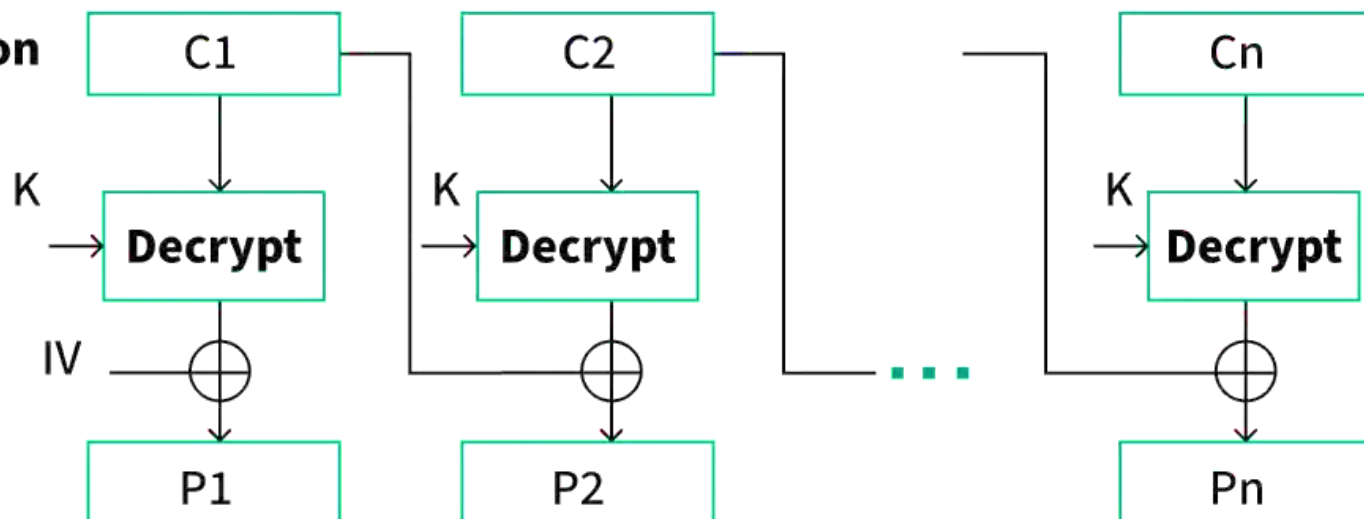
Cipher Block Chaining (CBC)

- Cipher block chaining or CBC is an **advancement made on ECB** since ECB compromises some security requirements.
- In CBC, the **previous cipher block** is given as **input** to the **next encryption algorithm** after **XOR** with the original **plaintext block**.
- Here, IV is a random or unique value used in combination with a secret key to initialize the encryption process.
- In a nutshell here, a cipher block is produced by encrypting an XOR output of the previous cipher block and present plaintext block.

Encryption



Decryption



Advantages:

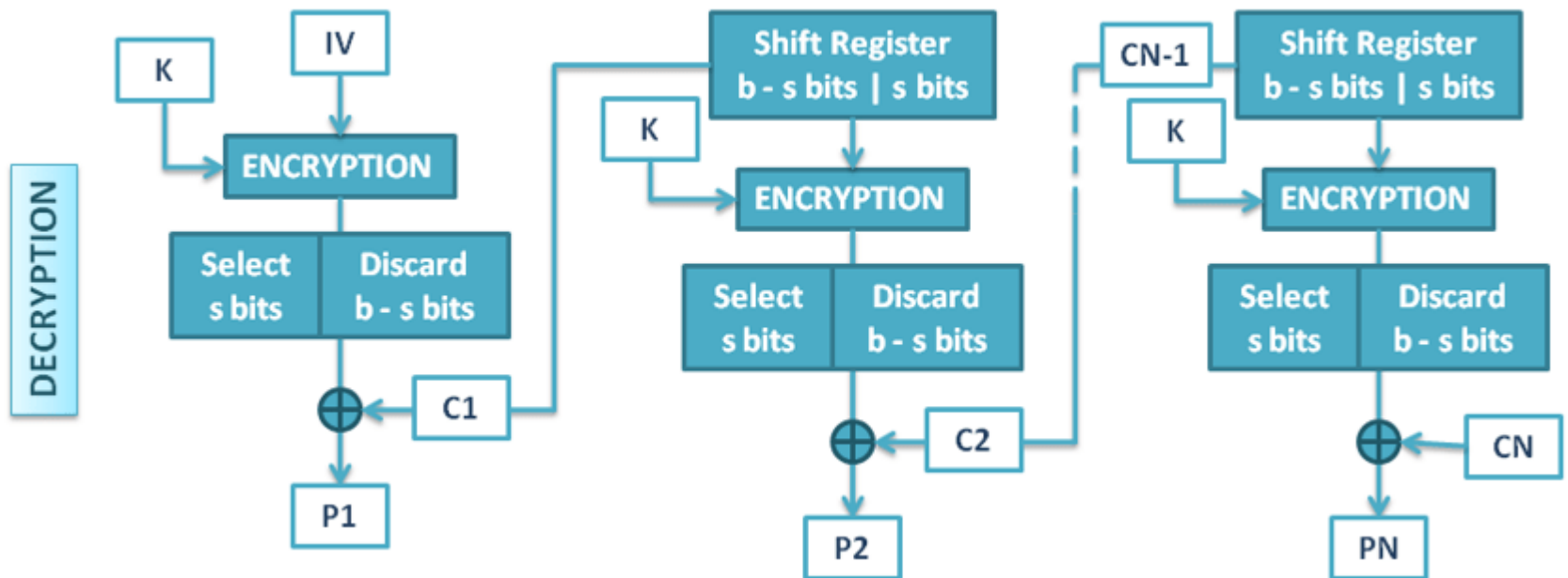
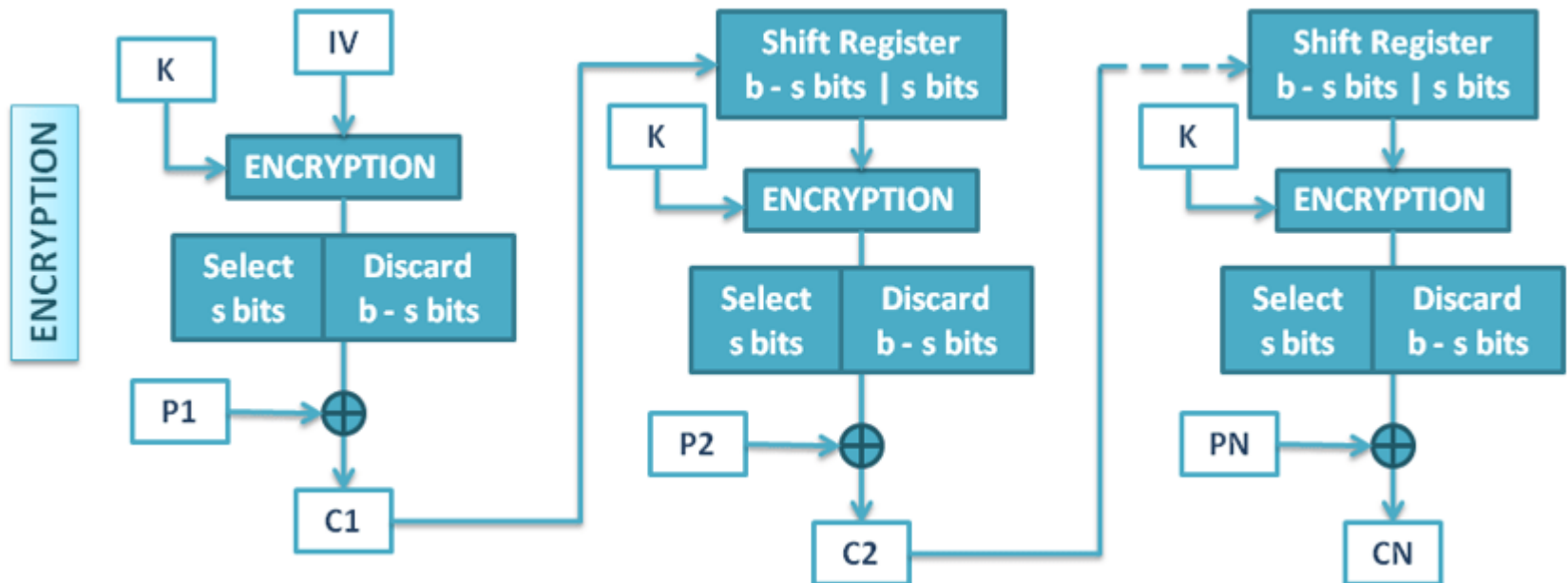
- CBC works well for input greater than b bits.
- CBC is a good authentication mechanism.
- Better resistive nature towards cryptanalysis than ECB.
- More secure than ECB as it hides patterns.

Disadvantages:

- Requires the previous cipher text block for encryption and decryption, making parallel processing difficult.

Cipher Feedback Mode (CFB)

- In this mode the **cipher is given as feedback to the next block** of encryption with some new specifications:
- first, an initial vector IV is used for first encryption and output bits are divided as a **set of s and $b-s$ bits**.
- The **left-hand side s bits** are **selected along with plaintext bits to which an XOR operation is applied**.
- The **result** is given as input to a **shift register** having **$b-s$ bits to LHS, s bits to RHS** and the process continues.



Advantages:

- Since, there is some data loss due to the use of shift register, thus it is difficult for applying cryptanalysis.
- Can handle data streams of any size.

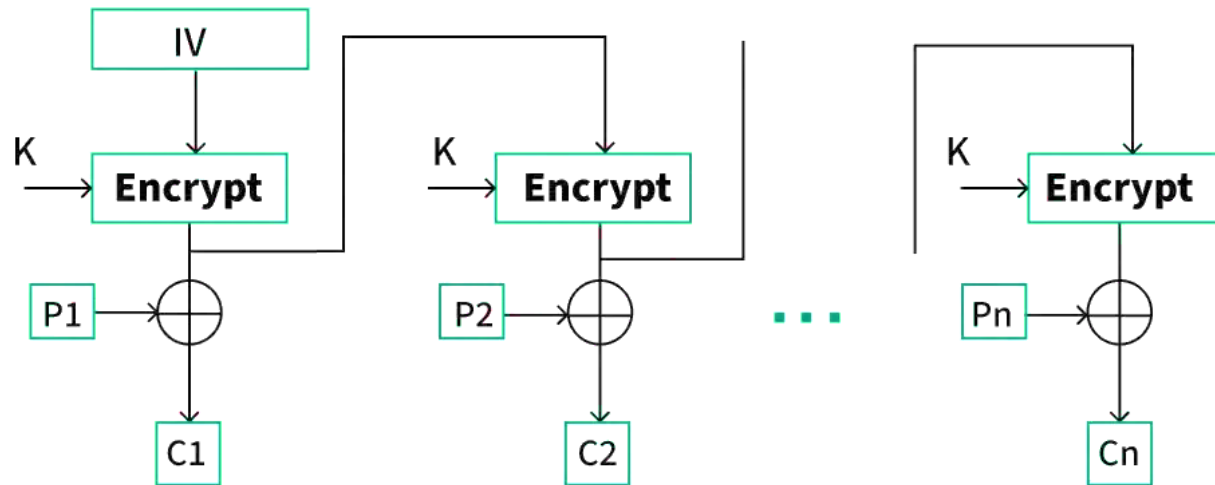
Disadvantages:

- The drawbacks of CFB are the same as those of CBC mode. Both block losses and concurrent encryption of several blocks are not supported by the encryption. Decryption, however, is parallelizable and loss-tolerant.
- Slightly more complex and can propagate errors.

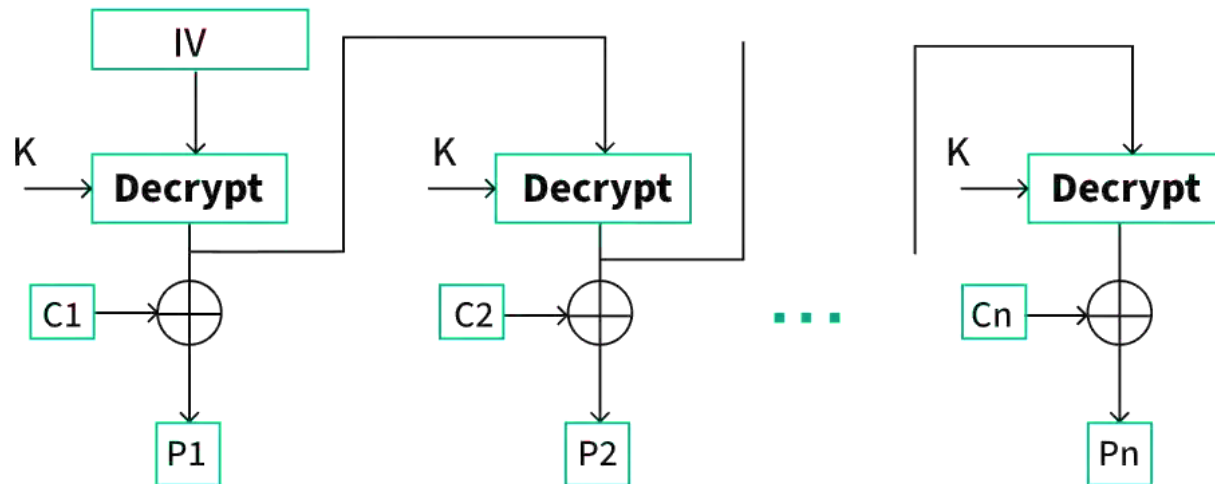
Output Feedback Mode(OFB)

- The output feedback mode follows nearly the same process as the Cipher Feedback mode except that it **sends the encrypted output as feedback instead of the actual cipher which is XOR output.**
- In this output feedback mode, all bits of the block are sent instead of sending selected s bits.
- The Output Feedback mode of block cipher holds **great resistance towards bit transmission errors.**
- It also **decreases the dependency or relationship of the cipher on the plaintext.**

Encryption



Decryption



Advantages:

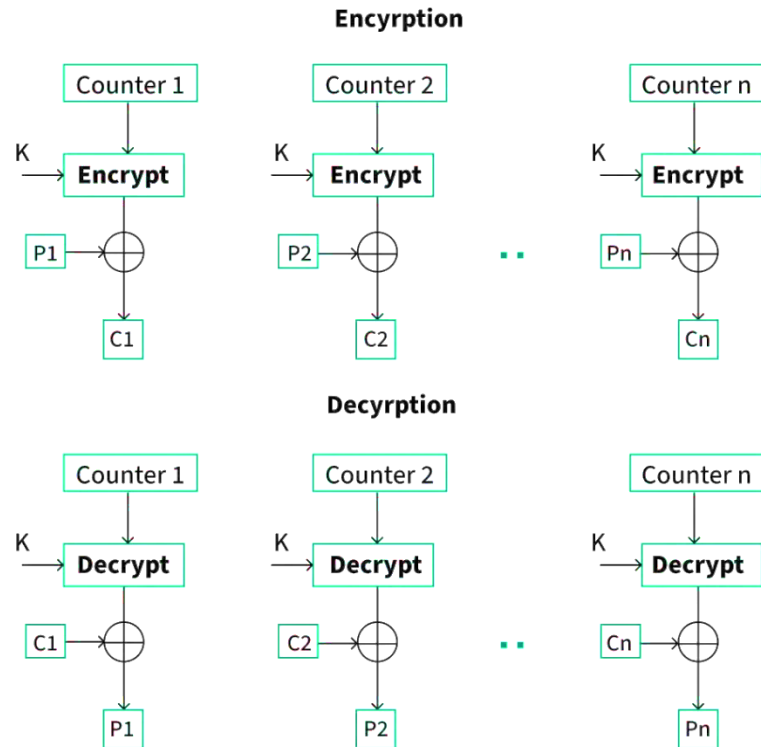
- In the case of CFB, a single bit error in a block is propagated to all subsequent blocks. This problem is solved by OFB as it is free from bit errors in the plaintext block. Thus **errors in transmission don't propagate**.

Disadvantages:

- The drawback of OFB is that, because to its operational modes, it is more susceptible to a message stream modification attack than CFB.
- If the key stream is reused, security is compromised.

Counter Mode(CTR)

- The Counter Mode or CTR is a simple **counter-based block cipher** implementation.
- Every time a counter-initiated value is encrypted and given as input to XOR with plaintext which results in cipher text block.
- The CTR mode is independent of feedback use and thus can be implemented in parallel.



Advantages:

- Since there is a different counter value for each block, the direct plaintext and cipher text relationship is avoided. This means that the same plain text can map to different cipher text.
- Parallel execution of encryption is possible as outputs from previous stages are not chained as in the case of CBC.

Disadvantages:

- The fact that CTR mode requires a synchronous counter at both the transmitter and the receiver is a severe drawback. The recovery of plaintext is inaccurate when synchronization is lost.

Applications of Block Ciphers

1. Data Encryption
2. File and Disk Encryption
3. Virtual Private Networks (VPN)
4. Secure Sockets Layer (SSL) and Transport Layer Security (TLS)
5. Digital Signatures

Other Modes of Operation

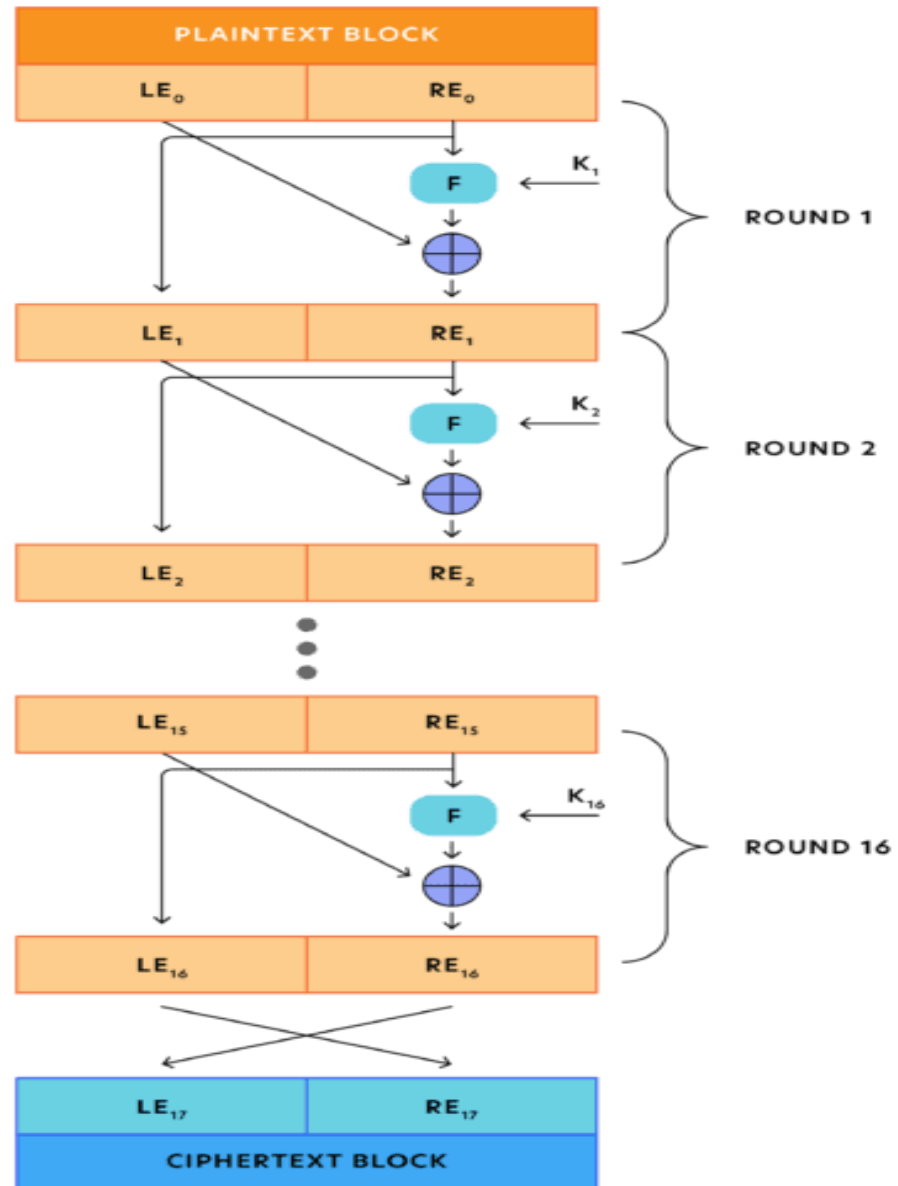
- GCM (Galois/Counter Mode)
- CCM (Counter with CBC-MAC)
- XTS (XEX Tweakable Block Cipher with Cipher text Stealing)
- EAX Mode
- OCB (Offset Codebook Mode)
- PCBC (Propagating Cipher Block Chaining)
- LRW (Liskov, Rivest, Wagner) Mode

Feistel Cipher Structure (S-P network)

- **Claude Shannon** introduced idea of **substitution-permutation (S-P) networks**.
Form basis of **modern block ciphers**
- S-P nets are based on the **two primitive cryptographic operations** seen before:
 - ***Substitution (s-box)***
 - ***Permutation (p-box)***
- Provide ***confusion & diffusion*** of message & key
- **Confusion and Diffusion:**
 - Confusion – makes relationship between ciphertext and key as complex as possible. Given CT, no Information about PT, Key, Encry. Algorithm etc., E.g: Substitution
 - Diffusion – dissipates statistical structure of plaintext over bulk of ciphertext. 1 bit change in PT, Significant Effect on CT. E.g: Transposition or Permutation

Feistel Cipher Structure:

- Horst Feistel devised the Feistel cipher Based on concept of **invertible product cipher**



- Input- plaintext block ($2w$ bits)
 - L_0 and R_0
- Process through **n-rounds** and **combine to produce cipher text**.
- Each round ' i ' has inputs L_{i-1} and R_{i-1} , which is derived from previous rounds and **subkey k_i** . (k_i derived from K).
- **Each k_i is different from each other.**
- **Round function has the same structure.**
- **Substitution** is performed on the **left half of the data**. This is done by applying round function F to R_i and taking XOR of the output of the function and the L_i .
- Round function has the same general structure and parameterized by the round subkey K_i

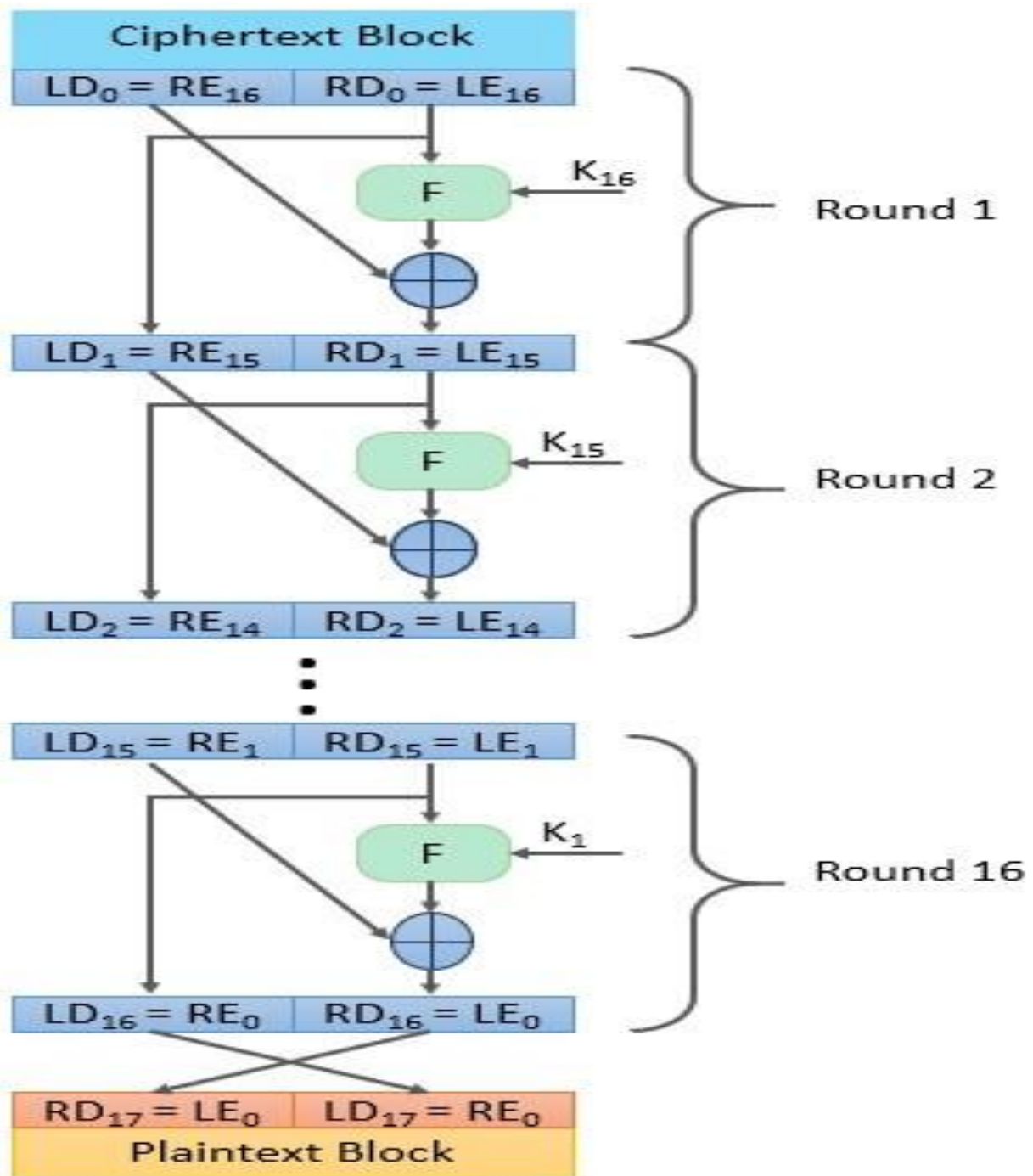
- Following the substitution, a permutation is performed by interchanging the two halves of the data.
- Implements **Shannon's S-P network concept**
- In particular, Feistel proposed the use of a cipher that alternates substitutions and permutations, where these terms are defined as follows:
 - **Substitution:** Each plaintext element or group of elements is uniquely replaced by a corresponding ciphertext element or group of elements.
 - **Permutation:** A sequence of plaintext elements is replaced by a permutation of that sequence. That is, no elements are added or deleted or replaced in the sequence, rather the order in which the elements appear in the sequence is changed

Feistel Cipher Design Elements:

- **Block size:** larger block size means greater security but reduces E/D speed.
- **Key size:** larger key size means greater security but reduces E/D speed.
- **Number of rounds:** single round offers inadequate security but that multiple rounds offers greater security. (16 rounds)
- **Subkey generation algorithm:** Greater complexity should lead to greater difficulty in cryptanalysis.
- **Round function:** greater complexity should lead to greater difficulty in cryptanalysis.
- **Fast software encryption/decryption**
- **Ease of analysis**

Feistel Decryption Algorithm:

- Same as the encryption process.
- Ciphertext as input, but use the **subkeys K_i in reverse order.**
- **K_n First round, K_{n-1} in second round and K_1 in last round.**
- So, we need not to implement two different algorithms for encryption and decryption.
- Notation for encryption LE_i and RE_i
- Notation for decryption LD_i and Rd_i



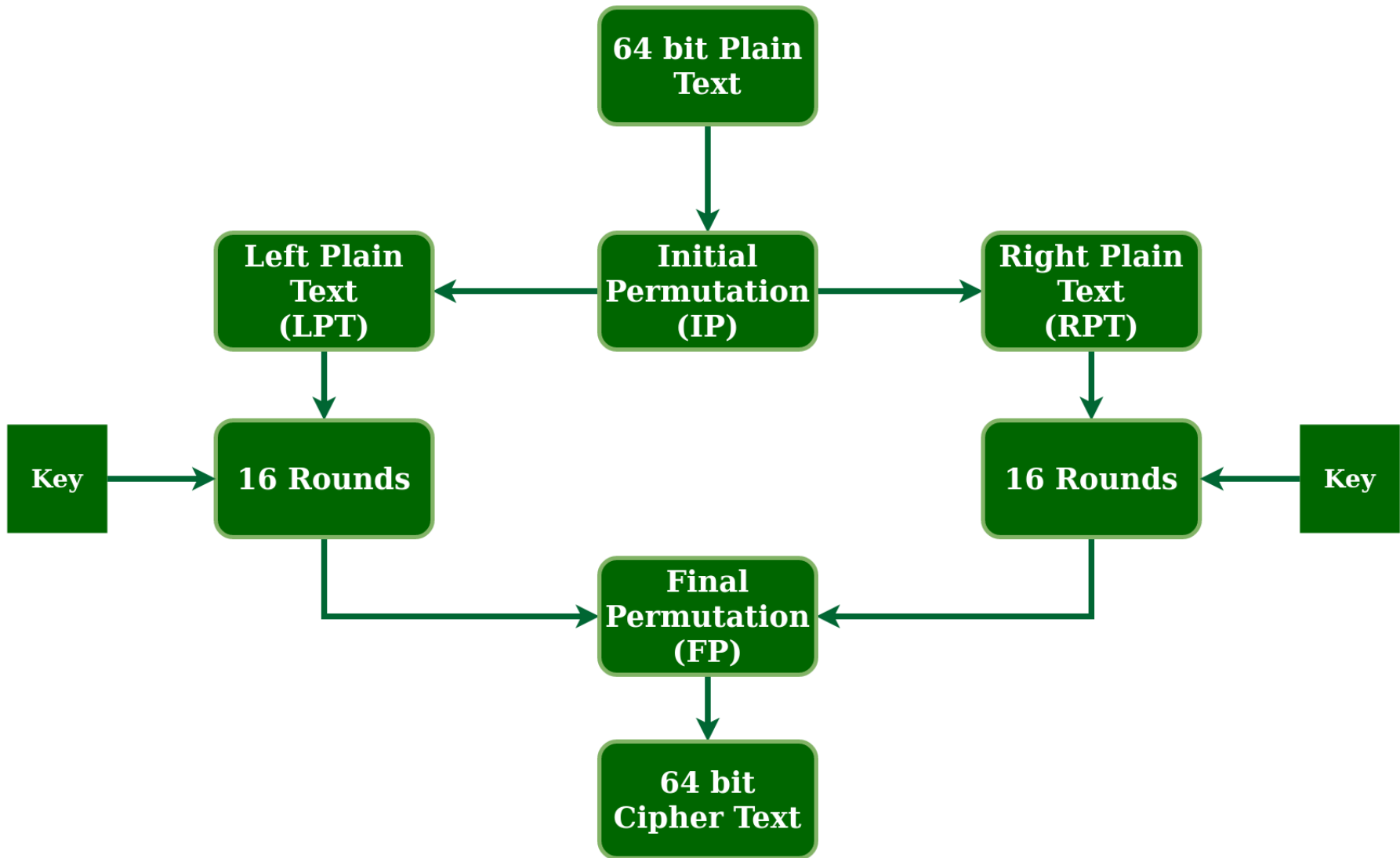
- Intermediate value of the decryption process is equal to the corresponding value of the encryption process with two halves the value swapped.
 - i.e., output of i th encryption is $LE_i || RE_i$
 - then, the corresponding $(16-i)$ th decryption round is
 - $RE_i || LE_i$ or $RD_{16-i} || LD_{16-i}$
- After the last iteration of encryption process two halves of the output are swapped, so the cipher text is $RE_{16} || LE_{16}$.(ciphertext)
- This cipher text is the input of the decryption algorithm.
- Input to the first round of the algorithm is $RE_{16} || LE_{16}$ which is equal to the 32bit swap of the algorithm.
- Output of the first round of the decryption process is equal to a 32-bit swap of the input to the sixteenth round of the encryption process.

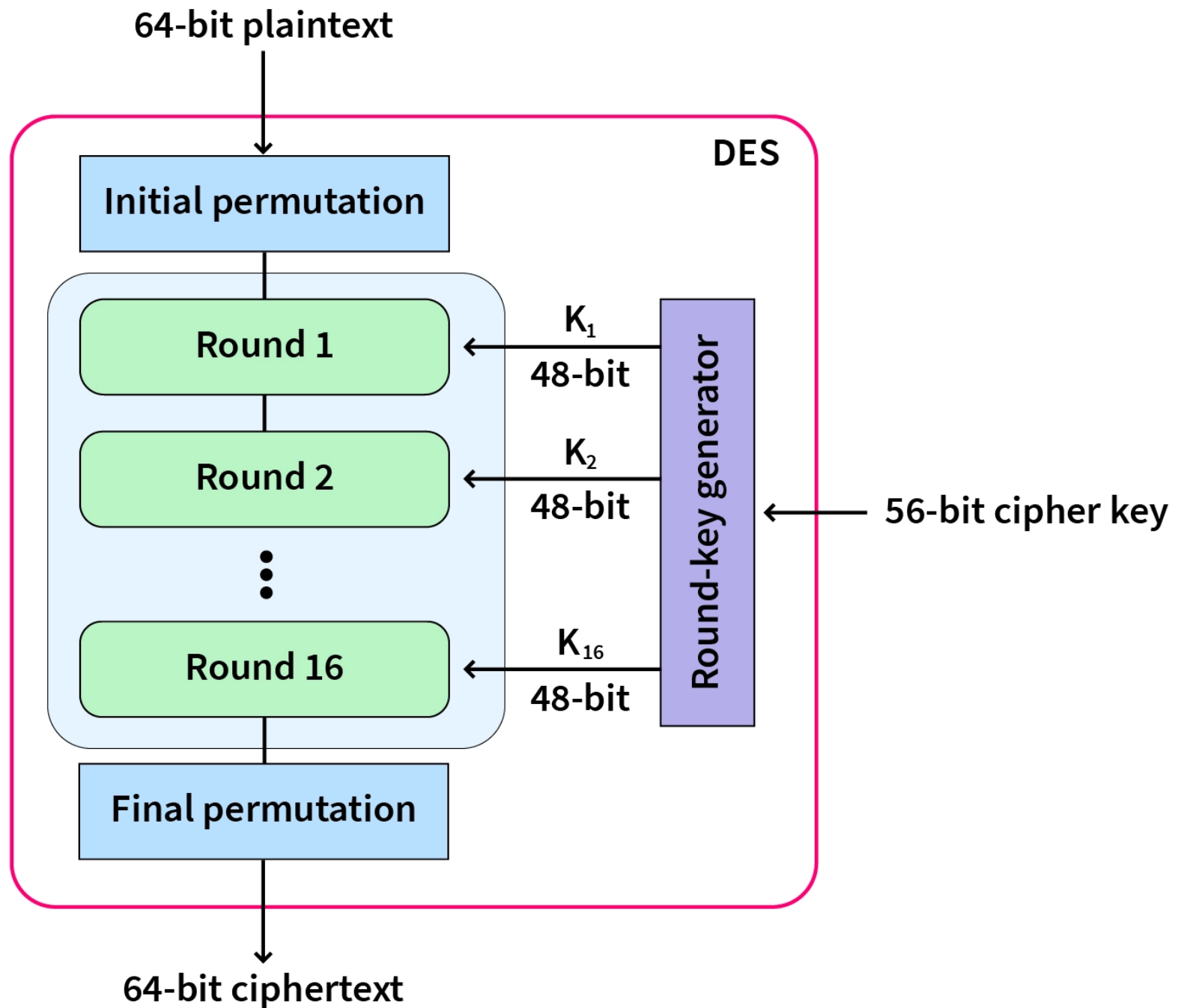
DES - DATA ENCRYPTION STANDARD

- The Data Encryption Standard (DES) is a **symmetric-key block cipher** published by the **National Institute of Standards and Technology (NIST)**.
- DES is an **implementation of a Feistel Cipher**. It uses **16 round Feistel structure**. The **block size is 64-bit**. Though, key length is 64-bit, DES has an effective **key length of 56 bits**, since 8 of the 64 bits of the key are not used by the encryption algorithm (function as check bits only).
- We have mentioned that DES uses a 56-bit key. Actually, The initial key consists of 64 bits. However, before the DES process even starts, every 8th bit of the key is discarded to produce a 56-bit key. That is bit positions 8, 16, 24, 32, 40, 48, 56, and 64 are discarded.

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32
33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48
49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64

- Thus, the discarding of every 8th bit of the key produces a **56-bit key** from the original **64-bit key**.
- DES is based on the two fundamental attributes of cryptography: **substitution (also called confusion)** and **transposition (also called diffusion)**.
- DES consists of **16 steps**, each of which is called a **round**. Each round performs the steps of substitution and transposition.
 - In the first step, the **64-bit plain text block** is handed over to an **initial Permutation (IP) function**.
 - The initial permutation is performed on plain text.
 - Next, the initial permutation (**IP**) produces **two halves** of the permuted block; saying **Left Plain Text (LPT)** and **Right Plain Text (RPT)**.
 - Now each LPT and RPT go through 16 rounds of the encryption process.
 - In the end, **LPT and RPT are rejoined** and a **Final Permutation (FP)** is performed on the **combined block**
 - The **result** of this process produces **64-bit ciphertext**.

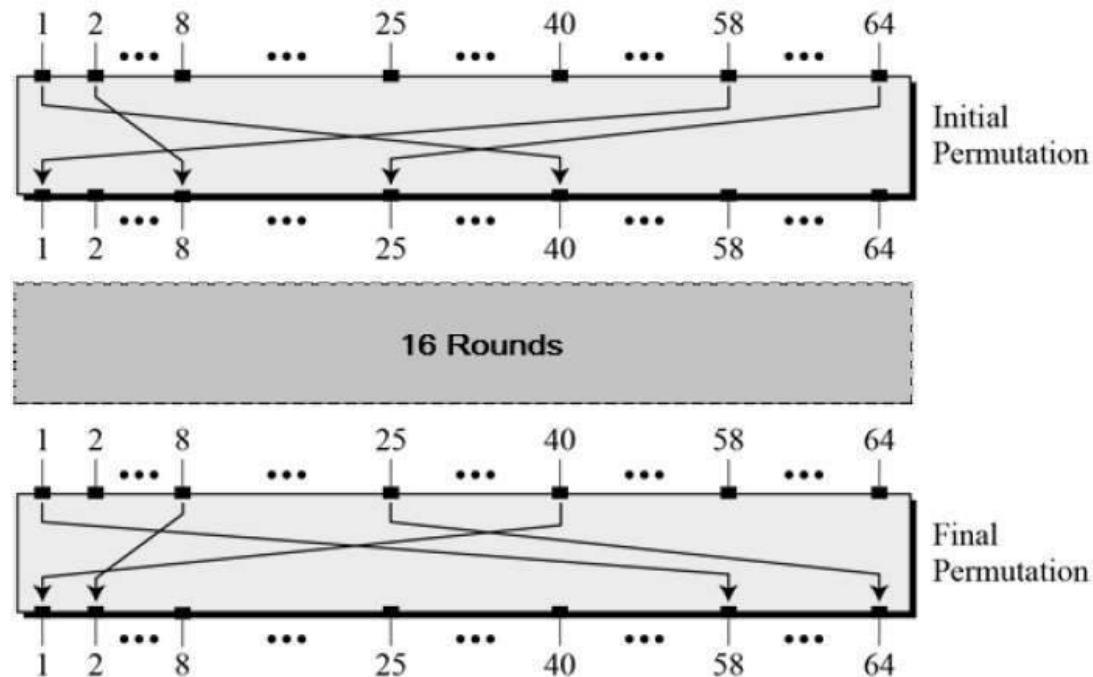




- Since DES is based on the Feistel Cipher, all that is required to specify DES is –
 - Round function
 - Key schedule
 - Any additional processing – Initial and final permutation

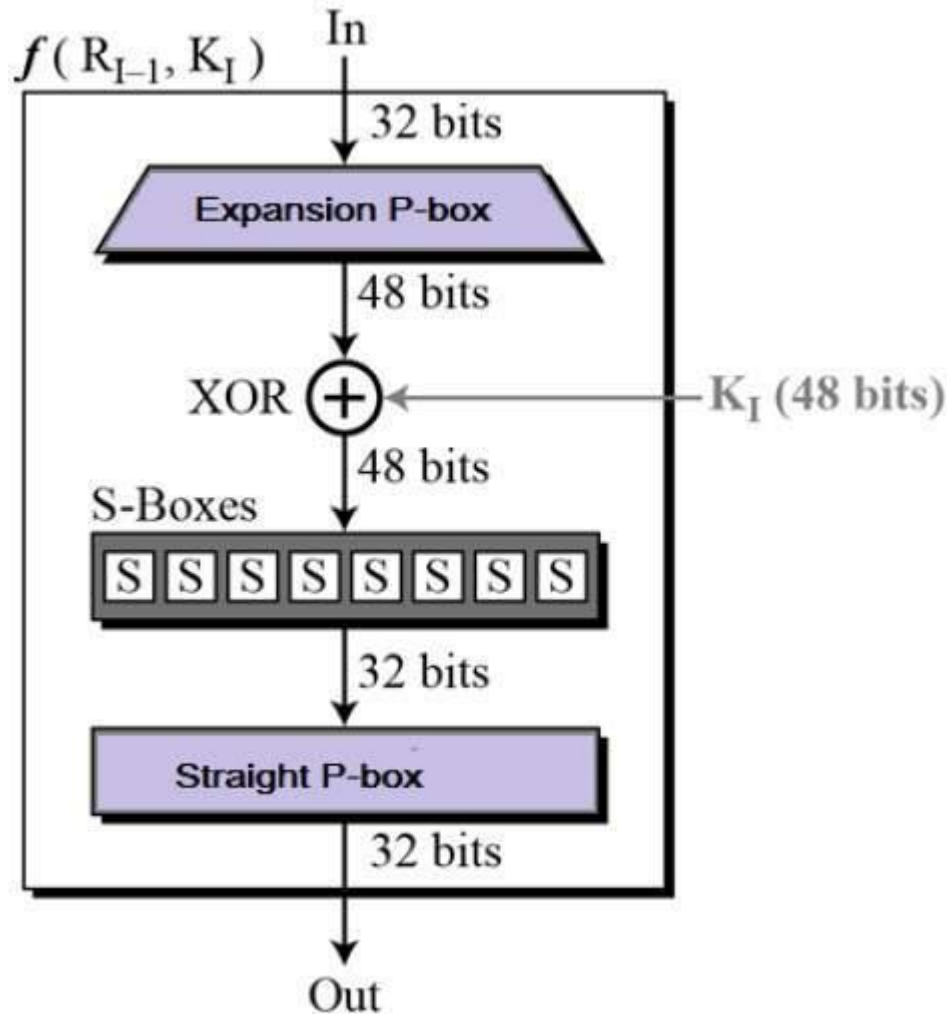
1. Initial and Final Permutation

- The initial and final permutations are straight Permutation boxes (P-boxes) that are inverses of each other.

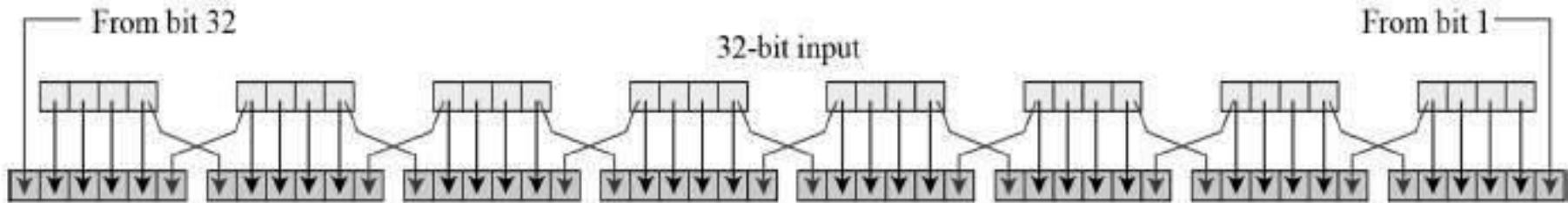


2. Round Function

- The heart of this cipher is the DES function, f .
- The DES function applies a 48-bit key to the rightmost 32 bits to produce a 32-bit output.



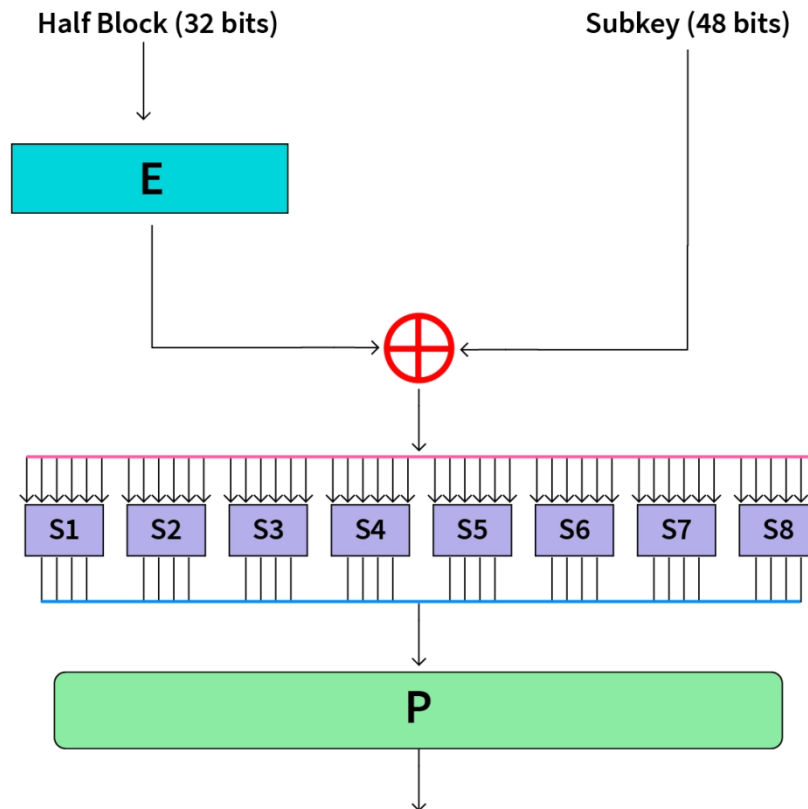
- **Expansion Permutation Box** – Since right input is 32-bit and round key is a 48-bit, we first need to expand right input to 48 bits. Permutation logic is graphically depicted in the following illustration.



- The graphically depicted permutation logic is generally described as table in DES specification illustrated as shown –

32	01	02	03	04	05
04	05	06	07	08	09
08	09	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	31	31	32	01

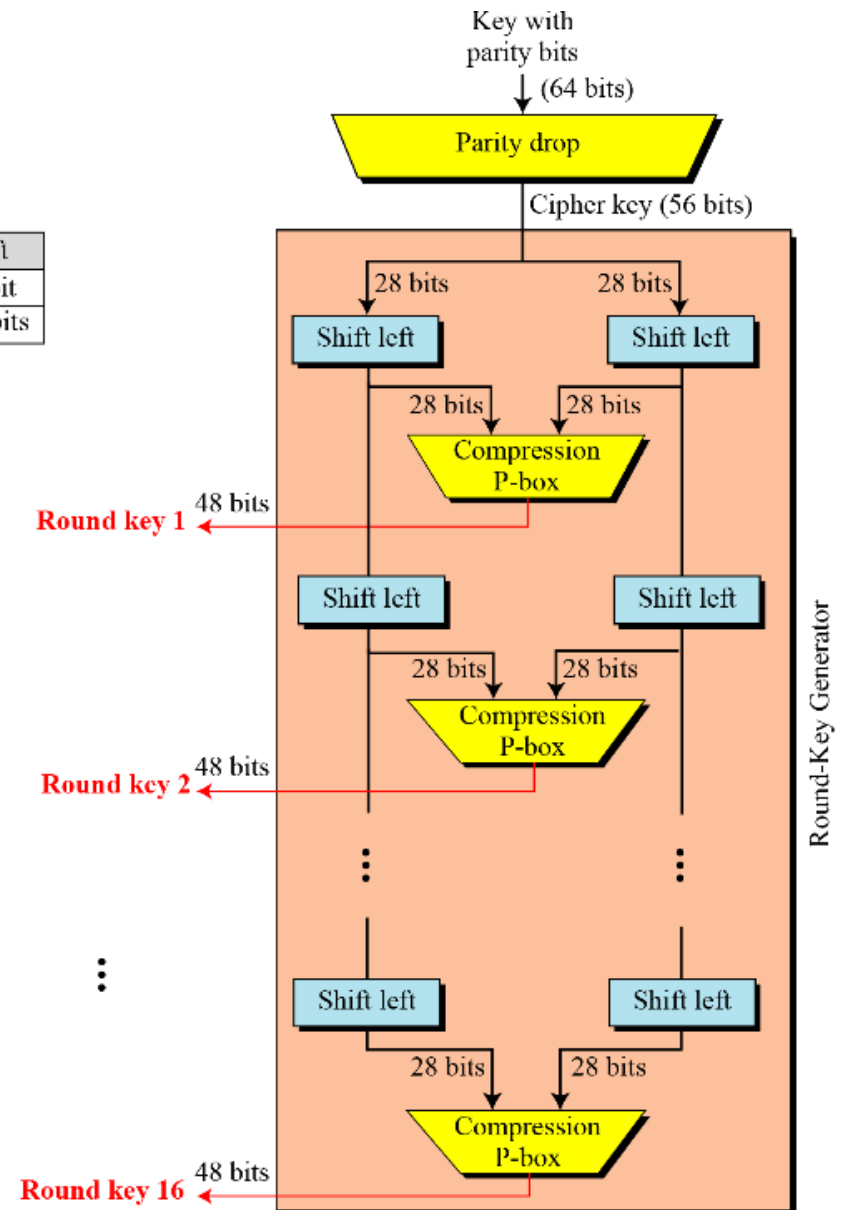
- **XOR (Whitener).** – After the expansion permutation, DES does XOR operation on the expanded right section and the round key. The round key is used only in this operation.
- **Substitution Boxes.** – The S-boxes carry out the real mixing (confusion). DES uses 8 S-boxes, each with a 6-bit input and a 4-bit output. Refer the following illustration –



3. Key Generation

- The round-key generator creates sixteen 48-bit keys out of a 56-bit cipher key.

Shifting	
Rounds	Shift
1, 2, 9, 16	one bit
Others	two bits



DES Analysis :

- **Avalanche effect** – A small change in plaintext results in the very great change in the ciphertext.
- **Completeness** – Each bit of ciphertext depends on many bits of plaintext.
- During the last few years, cryptanalysis have found some weaknesses in DES when key selected are weak keys. These keys shall be avoided.
- DES has proved to be a very well designed block cipher. There have been no significant cryptanalytic attacks on DES other than exhaustive key search.

AES-Advanced Encryption Standard

- Highly trusted encryption algorithm used to secure data by converting it into an unreadable format without the proper key.
- It is developed by the **National Institute of Standards and Technology (NIST) in 2001.**
- It is widely used today as it is much **stronger than DES and triple DES** despite being harder to implement.
- AES encryption uses **various key lengths (128, 192, or 256 bits)** to provide strong protection against unauthorized access.
- This data security measure is efficient and widely implemented in **securing internet communication, protecting sensitive data, and encrypting files.**
- AES, a cornerstone of modern cryptography, is recognized globally for its ability to keep information safe from cyber threats.

- AES is a Block Cipher.
- The key size can be 128/192/256 bits.
- Encrypts data in blocks of 128 bits each.

Working of The Cipher

- AES performs **operations on bytes of data rather than in bits.**
- Since the **block size is 128 bits**, the **cipher processes 128 bits (or 16 bytes)** of the input data at a time.
- The number of rounds depends on the key length as follows :

N (Number of Rounds)	Key Size (in bits)
10	128
12	192
14	256

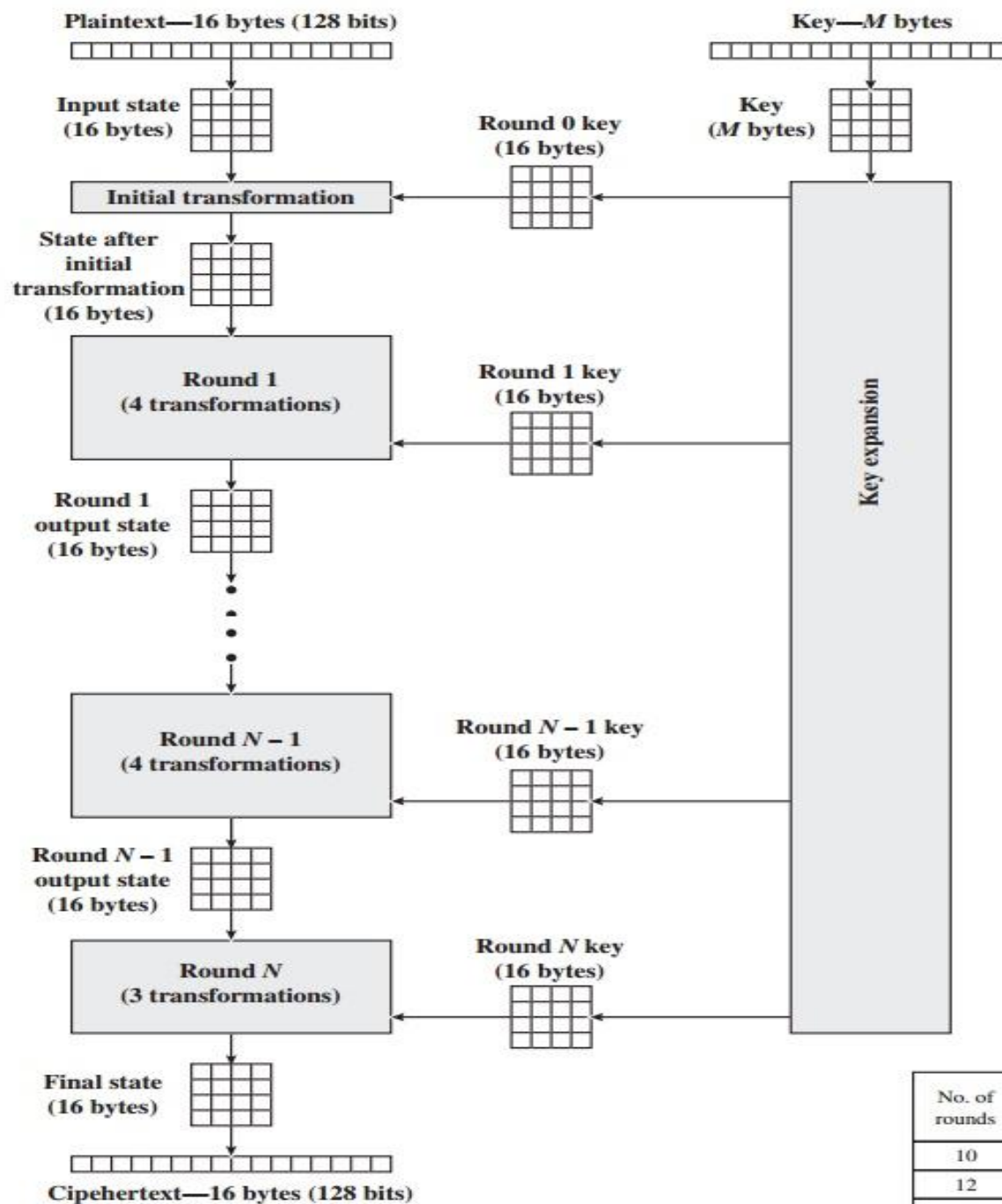
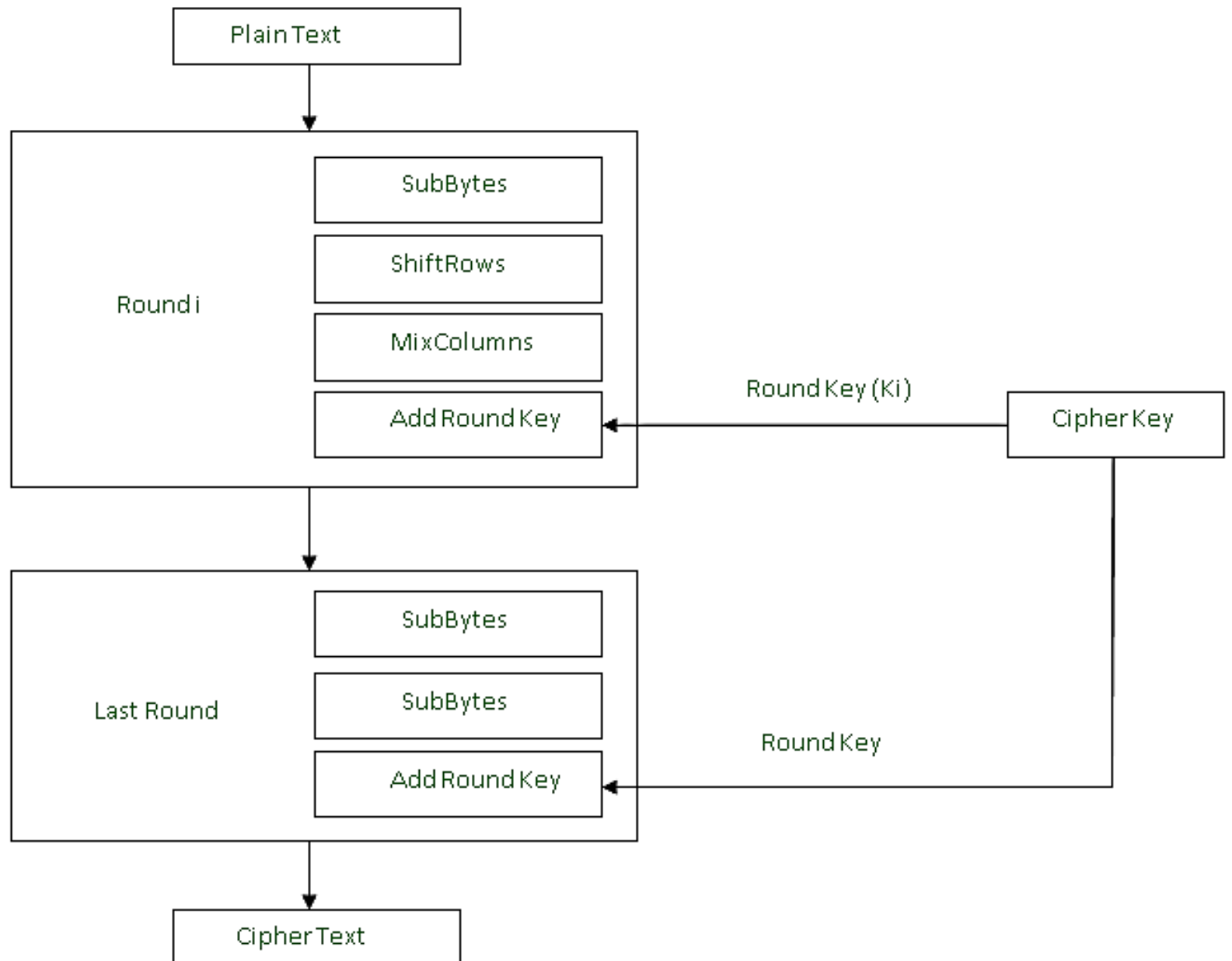


Figure 5.1 AES Encryption Process



Encryption

- AES considers each block as a 16-byte (4 byte x 4 byte = 128) grid in a column-major arrangement.

[	b0		b4		b8		b12	
	b1		b5		b9		b13	
	b2		b6		b10		b14	
	b3		b7		b11		b15	]

- Each round comprises of 4 steps :
 - **Sub Bytes**
 - **Shift Rows**
 - **Mix Columns**
 - **Add Round Key**

Step1. Sub Bytes

- This step implements the **substitution**.
- In this step, each byte is substituted by another byte.
- It is performed using a lookup table also called the **S-box**.
- This substitution is done in a way that a byte is never substituted by itself and also not substituted by another byte which is a compliment of the current byte.
- The result of this step is a 16-byte (4 x 4) matrix like before.
- The next two steps implement the permutation.

Step2. Shift Rows

- This step is just as it sounds. Each row is shifted a particular number of times.
- The first row is not shifted
- The second row is shifted once to the left.
- The third row is shifted twice to the left.
- The fourth row is shifted thrice to the left.
- (A left circular shift is performed.)

$[b_0 b_1 b_2 b_3]$		$[b_0 b_1 b_2 b_3]$
$ b_4 b_5 b_6 b_7 $	$\rightarrow$	$ b_5 b_6 b_7 b_4 $
$ b_8 b_9 b_{10} b_{11} $		$ b_{10} b_{11} b_8 b_9 $
$[b_{12} b_{13} b_{14} b_{15}]$		$[b_{15} b_{12} b_{13} b_{14}]$

Step 3: Mix Columns

- This step is a matrix multiplication.
- Each column is multiplied with a specific matrix and thus the position of each byte in the column is changed as a result.
- This step is skipped in the last round.

$$\begin{aligned} [c0] &= [2 \ 3 \ 1 \ 1] [b0] \\ |c1| &= |1 \ 2 \ 3 \ 1| |b1| \\ |c2| &= |1 \ 1 \ 2 \ 3| |b2| \\ [c3] &= [3 \ 1 \ 1 \ 2] [b3] \end{aligned}$$

Step 4: Add Round Keys

- Now the resultant output of the previous stage is XOR-ed with the corresponding round key. Here, the 16 bytes are not considered as a grid but just as 128 bits of data.
- After all these rounds 128 bits of encrypted data are given back as output. This process is repeated until all the data to be encrypted undergoes this process.

Decryption

- The stages in the rounds can be easily undone as these stages have an opposite to it which when performed reverts the changes. Each 128 blocks goes through the 10,12 or 14 rounds depending on the key size.
- The stages of each round of decryption are as follows :
 - **Add round key**
 - **Inverse MixColumns**
 - **ShiftRows**
 - **Inverse SubByte**

Applications of AES

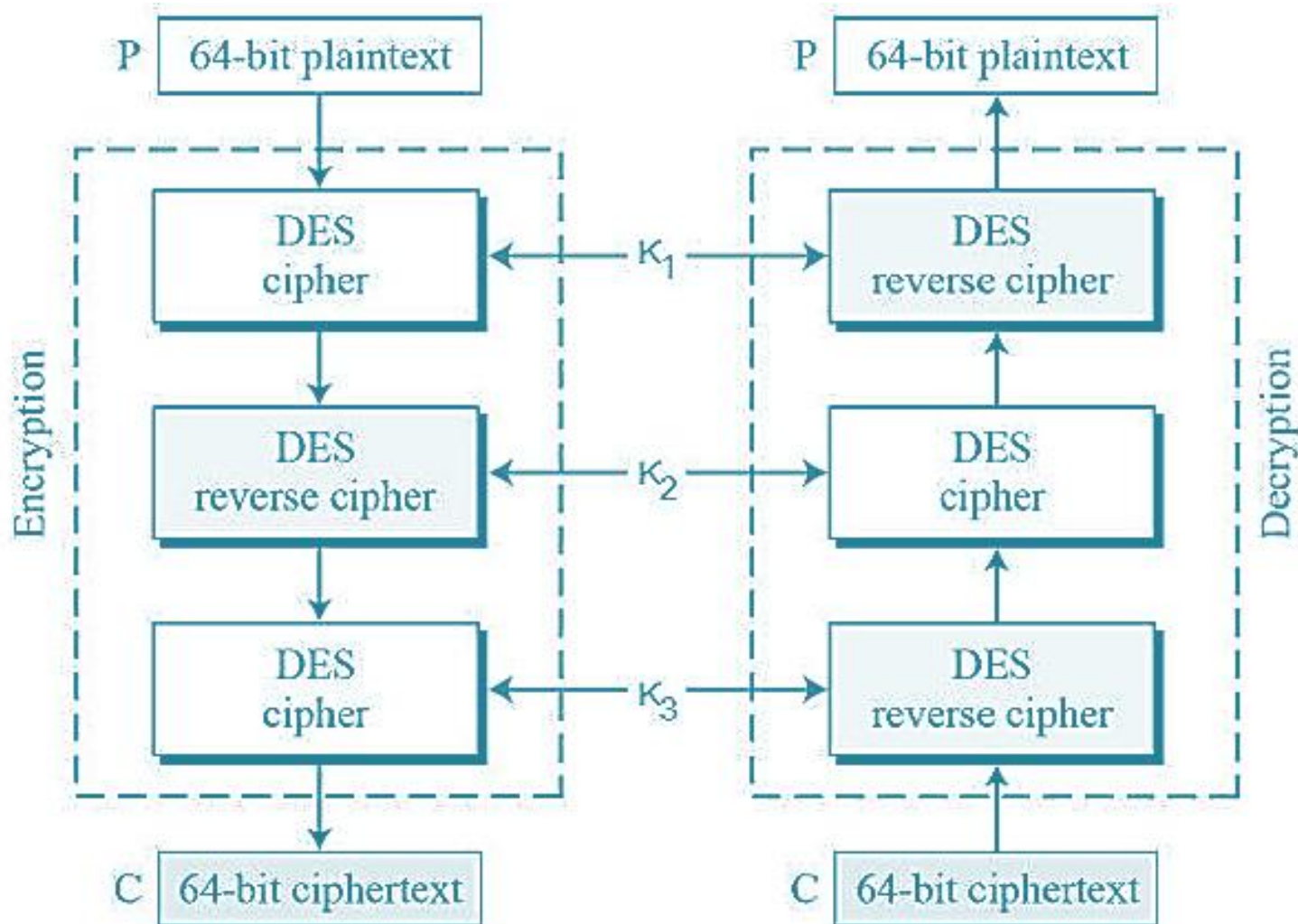
- **Wireless security:** AES is used in securing wireless networks, such as Wi-Fi networks, to ensure data confidentiality and prevent unauthorized access.
- **Database Encryption:** AES can be applied to encrypt sensitive data stored in databases. This helps protect personal information, financial records, and other confidential data from unauthorized access in case of a data breach.
- **Secure communications:** AES is widely used in protocols such as internet communications, email, instant messaging, and voice/video calls. It ensures that the data remains confidential.
- **Data storage:** AES is used to encrypt sensitive data stored on hard drives, USB drives, and other storage media, protecting it from unauthorized access in case of loss or theft.
- **Virtual Private Networks (VPNs):** AES is commonly used in VPN protocols to secure the communication between a user's device and a remote server. It ensures that data sent and received through the VPN remains private and cannot be deciphered by eavesdroppers.
- **Secure Storage of Passwords:** AES encryption is commonly employed to store passwords securely. Instead of storing plaintext passwords, the encrypted version is stored. This adds an extra layer of security and protects user credentials in case of unauthorized access to the storage.
- **File and Disk Encryption:** AES is used to encrypt files and folders on computers, external storage devices, and cloud storage. It protects sensitive data stored on devices or during data transfer to prevent unauthorized access.

Triple DES

- The speed of exhaustive key searches against DES after 1990 began to cause discomfort amongst users of DES.
- However, users did not want to replace DES as it takes an enormous amount of time and money to change encryption algorithms that are widely adopted and embedded in large security architectures.
- The pragmatic approach was not to abandon the DES completely, but to change the manner in which DES is used.
- This led to the modified schemes of **Triple DES (sometimes known as 3DES)**.
- Incidentally, there are two variants of Triple DES known as **3-key Triple DES (3TDES)** and **2-key Triple DES (2TDES)**.

3-KEY Triple DES

- Before using 3TDES, user first generate and distribute a 3TDES key K , which consists of three different DES keys K_1 , K_2 and K_3 .
- This means that the actual 3TDES **key** has length $3 \times 56 = 168$ bits.



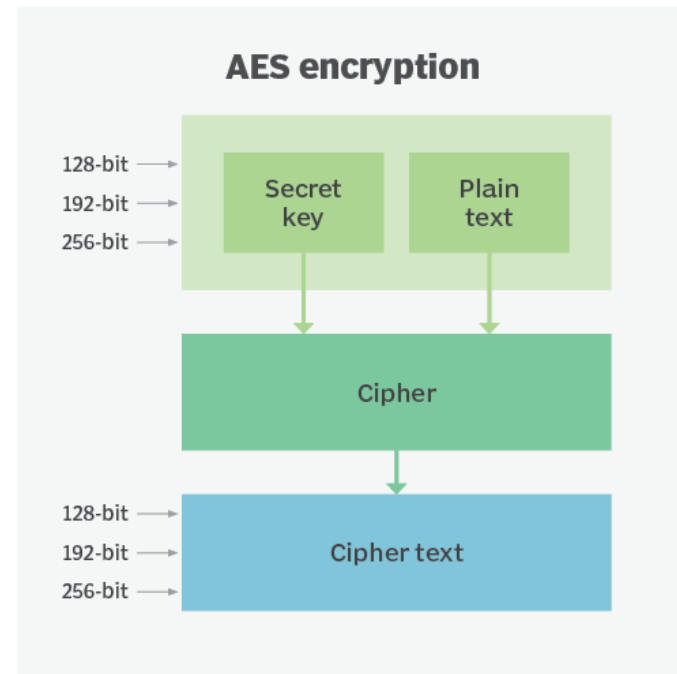
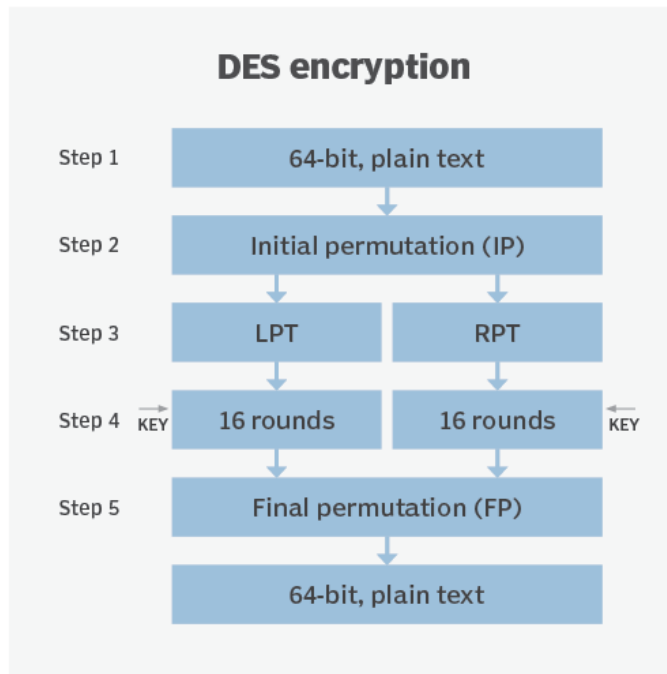
The encryption-decryption process is as follows –

- **Encrypt the plaintext blocks using single DES with key K1.**
- **Now decrypt the output of step 1 using single DES with key K2.**
- **Finally, encrypt the output of step 2 using single DES with key K3.**
- **The output of step 3 is the ciphertext.**
- **Decryption of a ciphertext is a reverse process. User first decrypt using K3, then encrypt with K2, and finally decrypt with K1.**
- Due to this design of Triple DES as an encrypt–decrypt–encrypt process, it is possible to use a 3TDES (hardware) implementation for single DES by setting K1, K2, and K3 to be the same value. This provides **backwards compatibility with DES.**
- Second variant of Triple DES (2TDES) is identical to 3TDES except that K3 is replaced by K1. In other words, user encrypt plaintext blocks with key K1, then decrypt with key K2, and finally encrypt with K1 again. Therefore, **2TDES has a key length of 112 bits.**
- Triple DES systems are significantly more secure than single DES, but these are clearly a much slower process than encryption using single DES.

Difference between AES and DES Ciphers

- Both DES and AES are symmetric-key block ciphers that are used in encryption where just one key (the secret key) is utilized to encode and decode electronic data. The key must be exchanged between the organizations communicating using symmetric encryption so that it can be utilized in the decryption process.

DES encryption vs. AES encryption



Key	AES	DES
Definition	AES stands for Advanced Encryption Standard.	DES stands for Data Encryption Standard.
Key Length	Key length varies from 128 bits, 192 bits to 256 bits.	Key length is of 56 bits.
Rounds of Operations	•Rounds per key length: •128 bits - 10 •192 bits - 12 •256 bits – 14	16 rounds of identical operations.
Network	AES structure is based on substitution-permutation network.	DES structure is based on Feistel network.
Security	AES is de-facto standard and is more secure than DES.	DES is weak, however 3DES (Triple DES) is more secure than DES.
Rounds	The operation rounds involved in AES encryption are Byte Substitution, Shift Row, Mix Column, and Key Addition.	Expansion, XOR operation with round key, Substitution, and Permutation are the rounds used in DES encryption
Size	AES can encrypt 128 bits of plain text.	DES can encrypt 64 bits of plain text.
Derived from	AES is derived from Square cipher.	DES is derived from Lucifer cipher.
Designed By	AES was designed by Vincent Rijmen and Joan Daemen.	DES was designed by IBM.
Known attacks	No known attacks.	Brute-force, Linear crypt-analysis and Differential crypt-analysis.
Encryption	AES can encrypt plain text up to 128 bits.	DES can encrypt 64 bits of plain text.